

Task1_QA_Report

I have picked testing website as <https://demo.realworld.show/>

Bug Report:

Title	Steps to reproduce	Expected vs actual	Severity	Suspected cause
Weak password acceptance	Open the sign-up page. Enter a username and email address. Use password as "1" and click sign up	Expected: The application should reject weak password and display strength requirements. Actual: The account was created successfully using a single value as "1" and able to access the application	High	Missing password strength validation and rules.
No email verification for signing up	Open the Sign-Up page and enter a new username, email address, password. Click Sign Up and check whether the application requests email verification before granting access.	Expected: The application should send a verification email and require the user to verify ownership of the email address before activating the account. Actual: The account is created immediately, and the user can access the application home page without verifying the email address.	Medium	Email verification workflow is not implemented
JSON web token	Once you login, right click and go to inspect and navigate to the application → Local storage → https://demo.realworld.show/	Expected: the authentication token should be stored in HTTP only cookie and they should be inaccessible to client-side scripts. Actual: In this, the token is stored and is visible in the browser	High	Insecure token storage mechanism
No rate limiting on login page	Attempted multiple failed logins by entering wrong password to observe the server response.	Expected: Account should implement logout after 3 failed attempts. Actual: unlimited login attempts were allowed without restriction.	High	Missing brute force attack protection
Token remains valid after logout	Log in to the application and open the developer tools to copy JWT of user A from local storage. Then, log out from the portal and open browser console by sending a request to /api/user using copied token and press enter from user B	Expected: The token should be invalidated after logout and not disclose user information. Actual: The token remained valid and successfully returned user account information after logout.	High	session invalidation mechanism

Root cause Analysis:

The issue I selected for root-cause analysis is No Rate Limiting on the Login Page. In this scenario, the application allows multiple failed login attempts without blocking and not warning the user. I tested this by entering the wrong password several times and accepting the attempts without any restriction. This is risky because an attacker can try many password combinations against a user account until the correct password is found. The main reason for this issue is that the login endpoint does not appear to have brute-force protection mechanisms like rate limiting, account logout, CAPTCHA verification, or temporary delays after repeated failures. This becomes even more dangerous because the application also accepts very weak passwords such as "1", making user accounts easier to compromise. To fix this issue, the backend should limit failed login attempts by blocking or slowing requests after 3 to 5 unsuccessful attempts from the same user account or IP address. The system should also display a message such as "Too many login attempts, please try again later," and it should log the failed activities for monitoring purposes and notify the security or analyst team to investigate suspicious behaviour. Implementing strong password policies together with rate limiting would significantly reduce the risk of brute-force attacks and improve the overall security of the application.